

MediVault – System Overview (Tech + Flow)

Generated: 2026-01-05

What this system is
MediVault is a full-stack healthcare management system (EHR-style).
One server serves both: frontend (React SPA) and backend API (/api/*).

Technology stack (high level)
Frontend: React + TypeScript + Vite + Tailwind + TanStack Query.
Backend: Node.js + Express + TypeScript.
Auth: Passport Local + server-side sessions stored in PostgreSQL.
Database: PostgreSQL + Drizzle ORM.
Realtime: WebSockets (scanner/real-time features).
Email: Nodemailer utilities.
AI (optional): Gemini integration.

Where the backend is (files/folders)
Backend folder: server/
Backend entry/startup: server/index.ts (creates app, sessions, registerRoutes, starts server)
API routes: server/routes.ts (most /api/* endpoints)
Auth routes + auth middleware: server/localAuth.ts (POST /api/login, isAuthenticated, role guards)
DB connection: server/db.ts (pg Pool + drizzle db)
DB operations layer (DB controllers): server/storage.ts (DatabaseStorage class)

Where DB schema is
Shared schema: shared/schema.ts (tables + types + Zod insert schemas)
Migrations: drizzle.config.ts + migration_*.sql files

How frontend calls the backend
Fetch helper: client/src/lib/queryClient.ts
Requests use fetch('/api/...', { credentials: 'include' }) so sessions work.
Pages mostly use TanStack Query queryKey: ['/api/...'].

Main API groups (examples)

Auth: POST /api/login (and related auth endpoints in server/localAuth.ts)

Profile: GET /api/profile, GET /api/profile/full

Patients: /api/patients, /api/patients/me, /api/patients/:id, /api/patients/nic/:nic

Doctors + availability: /api/doctors, /api/doctor-availability

Appointments: /api/appointments

Medical records/documents: /api/medical-records, /api/medical-documents

Prescriptions: /api/prescriptions, /api/prescriptions/doctor/mine

Lab: /api/lab-tests, /api/lab-facilities

Billing: /api/bills, /api/payments

Notifications: /api/notifications

Messaging: /api/conversations, /api/messages

Admin: /api/admin/* (users, stats, settings, backup/restore, reports)

System flow (simple)

1) User opens the UI (React) in the browser.

2) UI calls backend endpoints under /api/*.

3) Backend verifies session (cookie) + role guards.

4) Backend reads/writes PostgreSQL via Drizzle using shared/schema.ts.

5) Backend returns JSON; frontend updates UI and caches via TanStack Query.

6) Optional realtime: WebSocket server for scanner/real-time updates.